Digital Nurture 5.0

Python Full Stack Engineer Track

HANDS-ON EXERCISE BOOK

Python Backend Frameworks: Django, Flask & FastAPI

10 Hands-On Exercises

How to Use This Exercise Book

This exercise book covers Python Backend Frameworks for the Digital Nurture 5.0 Deep Skilling Program. The 10 hands-on exercises progressively build your backend development skills — starting from web framework foundations and Django, moving through Flask and FastAPI, and culminating in RESTful API design, authentication, and microservices architecture.

All exercises are built around a single scenario — a Course Management API — so each hands-on adds a new capability to the same system.

Software & Tools Required

- Python 3.10+ — https://www.python.org/downloads/
- VS Code — https://code.visualstudio.com/download
- pip packages: django, flask, fastapi, uvicorn, sqlalchemy, flask-sqlalchemy, pydantic, python-jose, passlib, bcrypt, httpx
- Postman or Thunder Client (VS Code extension) for API testing
- Git — https://git-scm.com/downloads
- Browser-based Python: Replit (replit.com) or PythonAnywhere (pythonanywhere.com)

Submission Guidelines

- Organise solutions in a folder named PythonBackendFrameworks/<YourName>/
- Each hands-on gets its own subfolder: handson_01/, handson_02/, ...
- Include a requirements.txt in each subfolder listing the packages used
- Push to your GitHub repository and share the URL with your POC

NOTE	Hands-On 1–3 use Django. Hands-On 4–5 use Flask. Hands-On 6–7 use FastAPI. Hands-On 8–10 are framework-agnostic — implement in the framework of your choice.

Difficulty Guide

Level	Hands-On	What to Expect
Beginner	1, 2, 3	Django setup, models, views, ORM, admin
Intermediate	4, 5, 6, 7	Flask routing, SQLAlchemy, FastAPI, async
Advanced	8, 9, 10	REST design, JWT auth, microservices concepts

Common Scenario: Course Management API

All exercises build one unified backend project — a Course Management API for a college system. Each hands-on adds a new layer: starting from a simple Django project, then recreating key features in Flask and FastAPI, and finally adding authentication and API design best practices.

> The Course Management API allows administrators to manage departments, courses, students, and enrollments. It exposes RESTful endpoints consumed by a frontend application. You will build this API three times — once each in Django, Flask, and FastAPI — to understand how each framework approaches the same problem differently.

Core API Endpoints (implement across all frameworks)

Endpoint	Method	Resource	Description
`/api/courses/`	GET	Courses	List all courses
`/api/courses/`	POST	Courses	Create a new course
`/api/courses/{id}/`	GET	Courses	Retrieve a course by ID
`/api/courses/{id}/`	PUT	Courses	Update a course
`/api/courses/{id}/`	DELETE	Courses	Delete a course
`/api/students/`	GET/POST	Students	List or create students
`/api/enrollments/`	POST	Enrollments	Enroll a student in a course
`/api/auth/login/`	POST	Auth	Login and receive JWT token

HANDS-ON 1 [Beginner]

Web Framework Foundations & Django Project Setup

Topics Covered:

✓ **Web Framework Concepts**	✓ **MVC / MVT Pattern**
✓ **Request-Response Cycle**	✓ **WSGI vs ASGI**
✓ **Django Project Setup**	✓ **URL Routing & Middleware**

Before writing any code, you need to understand what a web framework does and why. In this hands-on, you will explore framework fundamentals, then scaffold a Django project and understand its structure.

SET UP	pip install django then django-admin startproject coursemanager then cd coursemanager && python manage.py runserver

Task 1: Understand the Request-Response Cycle

Goal: Map out how a web framework handles an HTTP request from browser to response.

Steps:

1. Draw or describe (in comments inside a new file notes.py) the journey of a GET /api/courses/ request through a Django application: URL router → View → Model (DB query) → Response.
2. Identify where middleware sits in this cycle. Name two built-in Django middleware classes and describe what each does.
3. Explain the difference between WSGI and ASGI in comments. State which one Django uses by default and when you would switch to ASGI.
4. Explain the MVC pattern, then map it to Django's MVT (Model-View-Template): what does each letter correspond to in Django?

Hint:

- Django's 'View' does what a Controller does in MVC. The 'Template' is the View in MVC. The Model maps directly.

Expected Outcome: notes.py contains clear annotations for the request lifecycle, middleware role, WSGI vs ASGI, and the MVC-to-MVT mapping.

Task 2: Scaffold and Explore the Django Project

Goal: Create a Django project and a reusable app, and understand the generated file structure.

Steps:

5. Run django-admin startproject coursemanager to create the project. Inspect the generated files: settings.py, urls.py, wsgi.py, asgi.py. Write a one-line comment above each file explaining its role.
6. Run python manage.py startapp courses to create a Django app inside the project. Identify the difference between a Django project and a Django app.
7. Register the courses app in INSTALLED_APPS inside settings.py.
8. In courses/views.py, create a simple function-based view hello_view that returns an HttpResponse with 'Course Management API is running'.
9. In coursemanager/urls.py, add a URL pattern that maps /api/hello/ to hello_view.
10. Start the development server with python manage.py runserver and verify the endpoint at http://127.0.0.1:8000/api/hello/ returns the expected response.

Hint:

- A Django project is the overall configuration. A Django app is a self-contained module with its own models, views, and URLs — one project can have many apps.
- Use include() in the main urls.py to delegate to courses/urls.py — this keeps URL config modular.

Expected Outcome: Browser shows 'Course Management API is running' at /api/hello/. The app is listed in INSTALLED_APPS.

HANDS-ON 2 [Beginner]

Django Models, ORM & Admin Interface

Topics Covered:

✓ **Django Models**	✓ **Field Types & Constraints**
✓ **Migrations**	✓ **Django ORM Queries**
✓ **Admin Interface Registration**	

You will now define the data models for the Course Management system, run migrations to create database tables, query data using the Django ORM, and explore the built-in admin interface.

Task 1: Define Models and Run Migrations

Goal: Create Django models that represent the Course Management domain.

Steps:

11. In courses/models.py, define the following models: Department (name, head_of_dept, budget), Course (name, code — unique, credits, department — ForeignKey), Student (first_name, last_name, email — unique, department — ForeignKey, enrollment_year), Enrollment (student — ForeignKey, course — ForeignKey, enrollment_date, grade — nullable CharField).
12. Add __str__ methods to each model returning a human-readable string (e.g., return self.name for Course).
13. Run python manage.py makemigrations to generate the migration files. Inspect the generated migration file — identify the CreateTable operations.
14. Run python manage.py migrate to apply migrations. Confirm the tables exist by checking the database with python manage.py dbshell.
15. Add a Meta class to the Enrollment model with unique_together = [['student','course']] to prevent duplicate enrollments.

Hint:

- ForeignKey(Model, on_delete=models.CASCADE) means deleting a Department also deletes all its Courses — choose on_delete carefully based on business rules.
- Always run makemigrations after changing models, then migrate to apply — never edit migration files manually unless you know what you are doing.

Expected Outcome: python manage.py showmigrations shows all migrations as applied [X]. The dbshell shows the created tables.

Task 2: Django ORM Queries

Goal: Use the Django ORM shell to perform CRUD operations and complex queries.

Steps:

16. Open the Django shell with python manage.py shell. Create at least 2 Department, 4 Course, and 5 Student objects using Model.objects.create(...).
17. Query all courses in a specific department using Course.objects.filter(department__name='Computer Science'). Note the double underscore — this is a Django ORM lookup across a ForeignKey.
18. Use .values() and .annotate() to count the number of courses per department: Department.objects.annotate(course_count=Count('course')).
19. Use select_related to fetch all students along with their department in a single SQL query. Confirm with Django's connection.queries log.
20. Perform an update: increase the budget of all departments by 10% using Department.objects.update(budget=F('budget') * 1.1).

Hint:

- The double underscore (__) is Django ORM's way of traversing relationships and applying lookups — e.g., department__name performs a JOIN.
- F() objects let you reference field values in expressions without pulling data into Python — the calculation happens in the database.

Expected Outcome: Shell queries return expected results. The annotate query shows course counts per department. The F() update runs without fetching rows into Python.

Task 3: Django Admin Interface

Goal: Register models with the Django admin and explore its built-in management features.

Steps:

21. Create a superuser: python manage.py createsuperuser. Use admin / admin@college.edu / Admin@123 for local testing.
22. In courses/admin.py, register all four models: admin.site.register(Department), etc. Start the server and explore the admin at /admin/.
23. Customise the CourseAdmin: use list_display = ['name','code','credits','department'] to show multiple columns in the list view. Add search_fields = ['name','code'] to enable search.
24. Add list_filter = ['department'] to CourseAdmin to enable sidebar filtering by department.
25. Create 3 courses, 5 students, and 4 enrollments through the admin interface. Verify the unique_together constraint on Enrollment by trying to enroll the same student in the same course twice.

Hint:

- The admin interface is Django's superpower for rapid prototyping — you get full CRUD for all models with minimal code.
- ModelAdmin's list_display, search_fields, and list_filter are the three most impactful customisations for everyday use.

Expected Outcome: Admin lists courses with name, code, credits, department columns. Search and filter work. Duplicate enrollment raises a validation error.

HANDS-ON 3 [Beginner]

Django REST Views, URL Routing & Forms

Topics Covered:

✓ **Function-Based Views (FBV)**	✓ **Class-Based Views (CBV)**
✓ **URL Routing with include()**	✓ **Django REST Framework (DRF) basics**
✓ **Serializers**	✓ **Request & Response objects**

You will now build the actual REST API endpoints for the Course Management system using Django REST Framework — the most widely used package for building APIs in Django.

INSTALL	pip install djangorestframework then add 'rest_framework' to INSTALLED_APPS in settings.py

Task 1: Serializers and Basic API Views

Goal: Create DRF serializers and your first API views.

Steps:

26. In courses/serializers.py, create a ModelSerializer for each model: DepartmentSerializer, CourseSerializer, StudentSerializer, EnrollmentSerializer. Include all fields.
27. In courses/views.py, create a CourseListView using DRF's APIView: handle GET (return all courses serialized) and POST (create a new course from request.data).
28. Create a CourseDetailView for GET (single course by pk), PUT (update), and DELETE operations.
29. Wire both views in courses/urls.py: path('courses/', CourseListView.as_view()) and path('courses/<int:pk>/', CourseDetailView.as_view()). Include courses/urls.py in the main urls.py.
30. Test all endpoints using Postman or Thunder Client: GET /api/courses/ should return a JSON list; POST with a JSON body should create a course; DELETE /api/courses/1/ should remove it.

Hint:

- ModelSerializer auto-generates fields from the model — you only need to override for custom validation or computed fields.
- Return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST) when the serializer is invalid — never silently ignore validation errors.

Expected Outcome: All 5 HTTP methods (GET list, GET detail, POST, PUT, DELETE) work correctly and return appropriate status codes (200, 201, 204, 400, 404).

Task 2: ViewSets and Routers

Goal: Refactor views to use DRF ViewSets and automatic URL routing.

Steps:

31. Replace CourseListView and CourseDetailView with a single CourseViewSet that extends viewsets.ModelViewSet. This gives you all 5 CRUD operations with just 3 lines of code.
32. Create a DefaultRouter in courses/urls.py and register the viewset: router.register('courses', CourseViewSet). Include router.urls. Observe how the router auto-generates all URL patterns.
33. Do the same for StudentViewSet and EnrollmentViewSet.
34. Add a custom action to CourseViewSet using the @action decorator: a GET endpoint /api/courses/{id}/students/ that returns all students enrolled in that course.
35. Test the custom action endpoint in Postman. Verify it returns only students enrolled in the specified course.

Hint:

- ModelViewSet = ListModelMixin + CreateModelMixin + RetrieveModelMixin + UpdateModelMixin + DestroyModelMixin — all in one class.
- The router generates: GET/POST /courses/ and GET/PUT/PATCH/DELETE /courses/{pk}/ automatically.

Expected Outcome: GET /api/courses/ lists all courses. The custom /students/ action returns enrolled students for a specific course.

HANDS-ON 4 [Intermediate]

Flask — App Structure, Routing, Jinja2 & Blueprints

Topics Covered:

✓ **Flask App Structure**	✓ **Routing & URL Rules**
✓ **Jinja2 Templates**	✓ **Request & Response Objects**
✓ **Blueprints for Modular Design**	✓ **Flask Configuration**

Flask is a lightweight, unopinionated micro-framework. You will rebuild the Course Management API backend in Flask, appreciating how Flask's minimal approach differs from Django's batteries-included philosophy.

SET UP	pip install flask flask-sqlalchemy flask-migrate then create app.py as the entry point

Task 1: Flask App Structure and Basic Routing

Goal: Set up a Flask application with proper structure and define routes.

Steps:

36. Create a project folder flask_coursemanager/. Inside, create: app.py (entry point), config.py (configuration class), and a courses/ package with __init__.py and routes.py.
37. In app.py, create the Flask app using the application factory pattern: def create_app(): app = Flask(__name__); ... return app. This pattern makes the app testable.
38. In config.py, define a Config class with SQLALCHEMY_DATABASE_URI, SECRET_KEY, and DEBUG settings. Load config in create_app() using app.config.from_object(Config).
39. In courses/routes.py, define a Blueprint: courses_bp = Blueprint('courses', __name__, url_prefix='/api/courses'). Add route handlers for GET / and POST /.
40. Register the blueprint in create_app() with app.register_blueprint(courses_bp).
41. Run the app and test that GET /api/courses/ returns a JSON response using jsonify([]).

Hint:

- The application factory pattern (create_app function) is best practice in Flask — it avoids circular imports and makes testing easier.
- Blueprints are Flask's way of organising routes into modules — similar to Django apps but lighter.

Expected Outcome: GET /api/courses/ returns an empty JSON array. POST returns a 201 with the created course. The Blueprint is registered correctly.

Task 2: Request Handling and JSON Responses

Goal: Handle incoming request data and return properly structured JSON responses.

Steps:

42. In the POST / route, use request.get_json() to parse the request body. Validate that required fields (name, code, credits) are present — return 400 with an error message if any are missing.
43. Add a GET /<int:course_id>/ route, a PUT /<int:course_id>/ route, and a DELETE /<int:course_id>/ route in the blueprint.
44. Create a helper function make_response_json(data, status_code) that always returns a consistent JSON envelope: {'status': 'success', 'data': data}.
45. Add Flask error handlers for 404 and 500 using @app.errorhandler(404). Return JSON error responses (not HTML) — APIs should never return HTML error pages.
46. Use Postman to test all endpoints. Verify that missing fields return 400, unknown IDs return 404, and successful operations return 200 or 201.

Hint:

- request.get_json() returns None if the Content-Type header is not application/json — always check for None before accessing keys.
- Flask's default error responses are HTML. Register JSON error handlers so your API is consistent for all clients.

Expected Outcome: All endpoints return JSON. Missing fields trigger a 400 error with a descriptive message. Unknown IDs return 404.

HANDS-ON 5 [Intermediate]

Flask with SQLAlchemy ORM & Database Integration

Topics Covered:

✓ **Flask-SQLAlchemy Setup**	✓ **Model Definition**
✓ **Database Migrations with Flask-Migrate**	✓ **ORM CRUD Operations**
✓ **Relationship Queries**	✓ **Connection Pooling**

You will integrate SQLAlchemy into the Flask Course Management API, define models, run migrations, and perform all database operations through the ORM — connecting the API layer built in Hands-On 4 to a real database.

Task 1: Define SQLAlchemy Models and Migrations

Goal: Set up Flask-SQLAlchemy and define models matching the course management schema.

Steps:

47. In app.py, initialise Flask-SQLAlchemy: db = SQLAlchemy(). In create_app(), call db.init_app(app).
48. In courses/models.py, define Department, Course, Student, and Enrollment SQLAlchemy models using db.Model as the base. Mirror the same schema used in the Django hands-on.
49. Define relationships: Course has a db.relationship('Department', back_populates='courses'). Enrollment has relationships to both Student and Course.
50. Initialise Flask-Migrate: migrate = Migrate(app, db). Run flask db init, flask db migrate -m 'initial schema', flask db upgrade. Verify tables are created.
51. Use the Flask shell (flask shell) to insert 2 departments and 3 courses via the ORM. Commit using db.session.commit().

Hint:

- Always call db.session.commit() to persist changes. db.session.add(obj) stages the object but does not save it yet.
- flask db migrate only detects column additions/deletions reliably — relationship changes may need manual migration edits.

Expected Outcome: flask db upgrade creates all tables. Flask shell inserts work. db.session.query(Course).all() returns the inserted courses.

Task 2: Connect ORM to API Routes

Goal: Replace in-memory data in the Flask routes with real database queries.

Steps:

52. Update the GET /api/courses/ route to query the database: Course.query.all() (or db.session.execute(db.select(Course)).scalars()). Serialise results to a list of dicts using a to_dict() method on the model.
53. Add a to_dict() method to each model that returns a dictionary of its fields — this is Flask's equivalent of DRF serializers.
54. Update POST /api/courses/ to create a Course object, add it to the session, and commit. Return the new course as JSON with status 201.
55. Update GET /<int:id>/, PUT, and DELETE to use Course.query.get_or_404(id) — this automatically returns a 404 JSON response if the course is not found.
56. Add a route GET /api/courses/<int:id>/students/ that uses a JOIN query to return all students enrolled in the specified course.

Hint:

- to_dict() on models is a common Flask pattern. For larger projects, consider marshmallow or flask-marshmallow for serialization.

- Course.query.get_or_404(id) is the Flask-SQLAlchemy shortcut that combines a primary key lookup with a 404 abort in one line.

Expected Outcome: All CRUD endpoints read from and write to the database. The /students/ route returns the correct enrolled students via a JOIN.

HANDS-ON 6 [Intermediate]

FastAPI — Path Parameters, Pydantic & Async Endpoints

Topics Covered:

✓ **FastAPI Project Setup**	✓ **Path & Query Parameters**
✓ **Pydantic Models for Validation**	✓ **Async / Await in FastAPI**
✓ **Automatic OpenAPI / Swagger Docs**	✓ **Response Models**

FastAPI is a modern, high-performance framework built on ASGI that auto-generates OpenAPI documentation and uses Pydantic for data validation. You will rebuild the Course API in FastAPI and experience its developer-friendly features.

SET UP	pip install fastapi uvicorn sqlalchemy then uvicorn main:app --reload to run

Task 1: FastAPI Setup and Pydantic Schemas

Goal: Scaffold a FastAPI application and define Pydantic schemas for request/response validation.

Steps:

57. Create main.py with: from fastapi import FastAPI; app = FastAPI(title='Course Management API', version='1.0'). Add a root route @app.get('/') that returns {'message': 'API running'}.
58. Create schemas.py. Define Pydantic BaseModel classes: CourseCreate (name, code, credits, department_id), CourseUpdate (all fields optional using Optional[]), CourseResponse (all fields plus id — this is the response schema).
59. Define a DepartmentResponse schema that nests a list of CourseResponse objects to demonstrate Pydantic nested models.
60. In main.py, add a POST /api/courses/ endpoint with signature: async def create_course(course: CourseCreate). FastAPI automatically validates the request body against the Pydantic model and returns 422 if validation fails.
61. Visit http://127.0.0.1:8000/docs — observe the auto-generated Swagger UI showing your endpoints with request/response schemas.

Hint:

- Pydantic validates data at parse time — if the request body does not match the schema, FastAPI returns a detailed 422 Unprocessable Entity response automatically.
- Optional[str] = None marks a field as optional with a default of None. Use this in update schemas where not all fields are required.

Expected Outcome: /docs shows the POST /api/courses/ endpoint with the CourseCreate schema. Sending invalid data returns a 422 with field-level error details.

Task 2: Path/Query Parameters and Async Database Access

Goal: Add path and query parameters, and wire up async SQLAlchemy for database access.

Steps:

62. Add a GET /api/courses/{course_id} endpoint. The path parameter course_id is automatically validated as an integer by FastAPI.
63. Add a GET /api/courses/ endpoint with optional query parameters: skip: int = 0, limit: int = 10, department_id: Optional[int] = None. Use these to implement pagination and filtering.
64. Set up SQLAlchemy with an async engine using create_async_engine. Define a get_db() dependency function that yields a database session.
65. Inject the database session into endpoints using FastAPI's Dependency Injection: async def get_courses(db: AsyncSession = Depends(get_db)).

66. Implement the full CRUD for courses using async ORM queries: await db.execute(select(Course)), await db.commit(), etc.
67. Test pagination: GET /api/courses/?skip=0&limit=2 should return the first 2 courses; GET /api/courses/?skip=2&limit=2 should return the next 2.

Hint:

- FastAPI's Depends() system is its dependency injection mechanism — get_db() is called automatically before each request and cleaned up after.
- async def endpoints are handled by uvicorn's async event loop — they do not block other requests while waiting for I/O.

Expected Outcome: GET /api/courses/?limit=2 returns 2 courses. Filtering by department_id returns only courses in that department. All operations are async.

HANDS-ON 7 [Intermediate]

FastAPI — Dependency Injection, CRUD & OpenAPI Documentation

Topics Covered:

✓ **FastAPI Dependency Injection**	✓ **CRUD Operations**
✓ **Response Models & Status Codes**	✓ **Background Tasks**
✓ **OpenAPI Customisation**	✓ **Error Handling with HTTPException**

You will deepen your FastAPI skills by completing the full CRUD implementation, customising the OpenAPI documentation, handling errors properly, and using FastAPI's background tasks feature.

Task 1: Complete CRUD with Proper HTTP Conventions

Goal: Implement all CRUD endpoints following REST conventions for status codes and response models.

Steps:

68. Complete PUT /api/courses/{id} (update) and DELETE /api/courses/{id} (delete). Use response_model=CourseResponse on GET and POST endpoints to document and validate the response shape.
69. Use status_code=status.HTTP_201_CREATED on the POST endpoint and status_code=status.HTTP_204_NO_CONTENT on DELETE (no response body).
70. Use HTTPException(status_code=404, detail='Course not found') when a course ID does not exist. FastAPI converts this to a JSON error response automatically.
71. Add a GET /api/courses/{id}/students/ endpoint that returns all students enrolled in the course, using a JOIN query.
72. Implement all CRUD for Students and Enrollments following the same patterns.

Hint:

- response_model tells FastAPI what the response should look like — it filters out any extra fields from the ORM model that should not be exposed.
- HTTP 204 No Content is the correct status for successful DELETE — return nothing (just status code), not an empty JSON object.

Expected Outcome: DELETE returns 204 with no body. POST returns 201 with the created resource. Invalid IDs return 404 with a JSON detail message.

Task 2: Background Tasks and OpenAPI Customisation

Goal: Use FastAPI Background Tasks and customise the OpenAPI documentation.

Steps:

73. Add a BackgroundTasks parameter to the POST /api/enrollments/ endpoint. After creating an enrollment, add a background task that simulates sending a confirmation email: def send_confirmation_email(student_email: str): print(f'Sending confirmation to {student_email}').
74. Verify the endpoint returns immediately (201) without waiting for the background task — check the server console for the print output after the response.
75. Customise the OpenAPI metadata in the FastAPI() constructor: add title, description, version, and contact information.
76. Add tags to group related endpoints: @app.get('/api/courses/', tags=['Courses']). Observe how the Swagger UI organises endpoints by tag.
77. Add response_description and summary to the @app.post decorator for the create course endpoint. Check how this appears in the /docs UI.

Hint:

- Background tasks run after the response is sent — useful for non-critical work like sending emails or logging, but not for operations the client needs to wait for.

- OpenAPI tags and descriptions make your /docs page self-documenting — anyone can understand and test your API without reading the source code.

Expected Outcome: POST /api/enrollments/ returns 201 immediately. Server console shows the email confirmation print after the response. /docs shows grouped, described endpoints.

HANDS-ON 8 [Advanced]
RESTful API Design Best Practices

Topics Covered:

✓ **REST Principles**	✓ **HTTP Methods & Status Codes**
✓ **Resource Naming Conventions**	✓ **API Versioning**
✓ **Pagination & Filtering**	✓ **Error Response Standards**

Good API design is as important as working code. In this hands-on, you will audit and refactor your existing API endpoints to follow REST best practices — consistent naming, proper status codes, versioning, and standardised error responses.

NOTE	This hands-on applies to your Django, Flask, or FastAPI implementation. Pick one and refactor it to meet all the criteria below.

Task 1: Audit and Fix Resource Naming and HTTP Methods

Goal: Review existing endpoints and correct any REST convention violations.

Steps:

78. Review all your existing endpoints. Identify and fix any violations of REST resource naming: URLs should use nouns not verbs (/api/courses/ not /api/getCourses/), should be plural, and should use hyphens not underscores for multi-word resources.
79. Verify HTTP methods are semantically correct: GET (read, no side effects), POST (create), PUT (full replace), PATCH (partial update), DELETE (remove). Add a PATCH /api/courses/{id}/ endpoint for partial updates alongside the existing PUT.
80. Verify status codes: 200 OK for GET/PUT/PATCH, 201 Created for POST (with a Location header pointing to the new resource), 204 No Content for DELETE, 400 Bad Request for validation errors, 401 Unauthorised for missing auth, 404 Not Found, 422 Unprocessable Entity for schema errors.
81. Add a Location response header to all POST endpoints: response.headers['Location'] = f'/api/courses/{new_course.id}/'.

Hint:

- A good rule: if you can read the URL and know what resource it refers to without documentation, the naming is correct.
- PUT replaces the entire resource (all fields required). PATCH updates only the supplied fields (all fields optional). Use the right one.

Expected Outcome: All endpoints use plural nouns. PATCH is implemented alongside PUT. POST returns 201 with a Location header.

Task 2: Versioning, Pagination and Standardised Error Responses

Goal: Add API versioning, implement cursor/offset pagination, and standardise error formats.

Steps:

82. Add versioning to your API URLs: change /api/courses/ to /api/v1/courses/. Discuss (in a code comment) two alternative versioning strategies: URL versioning vs header-based versioning (Accept: application/vnd.api+json;version=1).
83. Implement offset pagination on GET /api/v1/courses/: accept page and page_size query params. Return a response envelope: {'count': total, 'next': url_or_null, 'previous': url_or_null, 'results': [...]}. This is the DRF pagination standard.
84. Add a filtering query parameter search= to GET /api/v1/courses/ that searches course name and code with a case-insensitive LIKE query.
85. Standardise all error responses to follow the format: {'error': {'code': 'NOT_FOUND', 'message': 'Course with id 99 does not exist', 'field': null}}. Update all error handlers to use this format.

Hint:

- URL versioning (/v1/) is simple and visible. Header versioning keeps URLs clean but is harder to test in a browser.
- Always include count and next/previous in paginated responses — clients need to know the total to build pagination controls.

Expected Outcome: GET /api/v1/courses/?page=1&page_size=2 returns the correct envelope with count, next, previous, and results. Error responses follow the standardised format.

HANDS-ON 9 [Advanced]

Authentication & Security — JWT, OAuth2 & OWASP

Topics Covered:

✓ **JWT Token Structure**	✓ **Token-Based Auth vs Session-Based Auth**
✓ **Password Hashing with bcrypt**	✓ **OAuth2 Flow (concept)**
✓ **CORS Configuration**	✓ **OWASP Top 10 Awareness**

Security is non-negotiable in any real API. In this hands-on, you will implement JWT-based authentication, protect routes, hash passwords, configure CORS, and review the most common web vulnerabilities from the OWASP Top 10.

INSTALL	pip install python-jose[cryptography] passlib[bcrypt] python-multipart

Task 1: Password Hashing and User Registration

Goal: Implement secure password storage using bcrypt hashing.

Steps:

86. Create a User model with fields: id, email (unique), hashed_password, is_active.
87. In a security.py utility file, implement: get_password_hash(password: str) -> str using passlib's CryptContext with bcrypt scheme, and verify_password(plain_password, hashed_password) -> bool.
88. Create a POST /api/v1/auth/register/ endpoint that: validates the email format, checks the email is not already registered (return 409 Conflict if so), hashes the password using get_password_hash, and saves the user.
89. Never store or log the plain-text password at any point. Add a comment in the code explaining why bcrypt is preferred over MD5 or SHA-256 for passwords.
90. Test: register a user, then inspect the database — confirm only the hashed password is stored, never the plain text.

Hint:

- bcrypt intentionally slow hashing (work factor) makes brute-force attacks computationally expensive — unlike MD5/SHA which are designed to be fast.
- Always check if the email is already registered before inserting — a UNIQUE constraint on the DB prevents duplicates, but returning a 409 gives the client a meaningful error.

Expected Outcome: Registration stores only the bcrypt hash. A second registration with the same email returns 409 Conflict.

Task 2: JWT Login, Protected Routes and CORS

Goal: Implement JWT token generation, route protection, and CORS configuration.

Steps:

91. Create a POST /api/v1/auth/login/ endpoint that: accepts email and password, verifies credentials using verify_password, creates a JWT token using python-jose with an expiry of 30 minutes, and returns {'access_token': token, 'token_type': 'bearer'}.
92. Create a get_current_user(token: str = Depends(oauth2_scheme)) dependency that decodes and validates the JWT token, returning the current user object. Raise 401 if the token is invalid or expired.
93. Protect the POST /api/v1/courses/ and DELETE /api/v1/courses/{id}/ endpoints by adding current_user: User = Depends(get_current_user) as a parameter. Unauthenticated requests should receive 401.
94. Configure CORS in your app to allow requests from http://localhost:3000 (your frontend dev server). In FastAPI: use CORSMiddleware. In Django: use django-cors-headers. In Flask: use flask-cors.

95. Document in comments: what is the OAuth2 Authorization Code flow, and how does it differ from the simple JWT login you just implemented?

Hint:

- JWT payloads are base64-encoded, not encrypted — never store sensitive data (passwords, credit cards) in a JWT payload.
- CORS headers are sent by the server to tell the browser which origins are allowed. CORS is enforced by the browser, not the server — server-side APIs are not protected by CORS.

Expected Outcome: Login returns a valid JWT. GET /api/v1/courses/ works without auth. POST /api/v1/courses/ returns 401 without a valid token. CORS allows localhost:3000.

HANDS-ON 10 [Advanced]

Microservices Architecture — Concepts & Decomposition

Topics Covered:

✓ **Monolith vs Microservices**	✓ **Service Decomposition**
✓ **Inter-Service Communication**	✓ **API Gateway Pattern**
✓ **Service Discovery (concept)**	✓ **Practising with Flask Microservices**

In this final hands-on, you will understand why and when to break a monolith into microservices, decompose the Course Management system into services, implement basic inter-service communication, and understand the API Gateway pattern.

Task 1: Decompose the Monolith into Services

Goal: Analyse the Course Management API and identify natural microservice boundaries.

Steps:

96. Review the Course Management API you built. Identify 3–4 bounded contexts — groups of related functionality that could be independent services. Document them in a README.md as: Service Name | Responsibility | Endpoints it owns | Database it owns.
97. The natural decomposition: Student Service (student CRUD, enrollment), Course Service (department and course CRUD), Auth Service (registration, login, token validation), Notification Service (email confirmations).
98. Create two minimal Flask apps in separate folders: course_service/ (port 5001) and student_service/ (port 5002). Each has its own database and its own subset of the original endpoints.
99. Verify each service runs independently: python app.py in each folder. Confirm they do not share a database.

Hint:

- The key microservices principle: each service owns its data. No service should directly query another service's database.
- Start with 2 services for this exercise — do not over-engineer. Microservices add operational complexity; justify each split with a clear business or scaling reason.

Expected Outcome: Two Flask apps run on separate ports. Each has its own SQLite database. Their endpoints do not overlap.

Task 2: Inter-Service Communication and API Gateway Pattern

Goal: Implement synchronous inter-service calls and understand the API Gateway pattern.

Steps:

100. Add an enrollment endpoint to Student Service: POST /api/students/{id}/enroll. This endpoint needs to verify the course exists — it must call Course Service's GET /api/courses/{id}/ using Python's requests library (pip install requests).
101. Handle the scenario where Course Service is unavailable: catch the ConnectionError and return 503 Service Unavailable with a descriptive message.
102. Create a simple API Gateway using Flask (gateway/ folder, port 5000): a single Flask app that proxies requests to the correct service. /api/courses/* → Course Service, /api/students/* → Student Service. Use requests.request() to forward calls.
103. Test the full flow through the gateway: POST http://localhost:5000/api/students/1/enroll with a course_id — the gateway routes to Student Service, which calls Course Service to verify the course.
104. Document in a README: what are the trade-offs of synchronous (HTTP) vs asynchronous (message queue) inter-service communication? When would you use a message queue like RabbitMQ or Kafka instead?

Hint:

- Synchronous inter-service calls create tight coupling — if Course Service is down, enrollment fails. Message queues decouple services at the cost of eventual consistency.
- A real API Gateway also handles auth, rate limiting, and SSL termination — your simple proxy demonstrates the routing concept without those production concerns.

Expected Outcome: POST to the gateway successfully routes through Student Service → Course Service. Stopping Course Service causes the enrollment endpoint to return 503.